Unicode in the Library, Part 2: Normalization

Document #:	P2729R0
Date:	2022-11-20
Project:	Programming Language C++
Audience:	SG-16 Unicode LEWG-I LEWG

Reply-to:	Zach Laine <whatwasthataddress@gmail.com>

Contents

§ 1 Motivation

I'm proposing normalization interfaces that meet certain design requirements that I think are important; I hope you'll agree:

— Ranges are the future. We should have range-friendly ways of doing transcoding. This includes support for sentinels and lazy views.

— Iterators are the present. We should support generic programming, whether it is done in terms of pointers, a particular iterator, or an iterator type specified as a template parameter.

— A null-terminated string should not be treated as a special case. The ubiquity of such strings means that they should be treated as first-class strings.

— If there's a specific algorithm specialization that operates directly on UTF-8 or UTF-16, the top-level algorithm should use that when appropriate. This is analogous to having multiple implementations of the algorithms in `std` that differ based on iterator category.

— Input may come from UTF-8, UTF-16, or UTF-32 strings (though UTF-32 is extremely uncommon in practice). There should be a single overload of each normalization function, so that the user does not need to change code when the input is changed from UTF-N to UTF-M. The most optimal version of the algorithm (processing either UTF-8 or UTF-16) will be selected (as mentioned above).

— The Unicode algorithms are low-level tools that most C++ users will not need to touch, even if their code needs to be Unicode-aware. C++ users should also be provided higher-level, string-like abstractions (provisionally called `std::text`) that will handle all the messy Unicode details, leaving C++ users to think about their program instead of Unicode).

§ 2 The shortest Unicode normalization primer I can manage

You can have different strings of code points that mean the same thing. For example, you could have the code point “ä” (U+00E4 Latin Small Letter A with Diaeresis), or you could have the two code points “a” (U+0061 Latin Small Letter A) and “¨” (U+0308 Combining Diaeresis). The former represents “ä” as a single code point, the latter as two. Unicode rules state that both strings must be treated as identical.

To make such comparisons more efficient, Unicode has normalization forms. If all the text you ever compare is in the same normalization form, it doesn’t matter whether they’re all the composed form like “ä” or the decomposed form like “a¨” – they’ll all compare bitwise-equal to each other if they represent the same text.

There are four official normalization forms. The first two are NFC (“Normalization Form: Composed”), and NFD (“Normalization Form: Decomposed”). There are these two other forms NFKC and NFKD that you can safely ignore; they are seldom-used variants of NFC and NFD, respectively.

NFC is the most compact of these four forms. It is near-ubiquitous on the web, as W3C recommends that web sites use it exclusively.

There’s this other form FCC, too. It’s really close to NFC, except that it is not as compact in some corner cases (though it is identical to NFC in most cases). It’s really handy when doing something called collation, which is not yet proposed. It’s coming, though.

§ 3 The stream-safe format

Unicode text often contains sequences in which a noncombining code point (e.g. ‘A’) is followed by one or more combining code points (e.g. some number of umlauts). It is valid to have an ‘A’ followed by 100 million umlauts. This is valid but not useful. Unicode specifies something called the Stream-Safe Format. This format inserts extra code points between combiners to ensure that there are never more than 30 combiners in a row. In practice, you should never need anywhere near 30 to represent meaningful text.

Long sequences of combining characters create a problem for algorithms like normalization and grapheme breaking; the grapheme breaking algorithm may be required to look ahead a very long way in order to determine how to handle the current grapheme. To address this, Unicode allows a conforming implementation to assume that a sequence of code points contains graphemes of at most 31 code points. This is known as the Stream-Safe Format assumption. All the proposed interfaces here and in the papers to come make this assumption.

The stream-safe format is very important. Its use prevents the Unicode algorithms from having to worry about unbounded-length graphemes. This in turn allows the Unicode algorithms to use side buffers of a small and fixed size to do their operations, which obviates the need for most memory allocations.

For more info on the stream-safe format, see the appropriate part of UAX15.

§ 3.1 Unicode reference

See UAX15 Unicode Normalization Forms for more information on Unicode normalization.

§ 4 Use cases

§ 4.1 Case 1: Normalize a sequence of code points to NFC

We want to make a normalized copy of s, and we want the underlying implementation to use a specialized UTF-8 version of the normalization algorithm. This is the most flexible and general-purpose API.

```
std::string s = /* ... */; // using a std::string to store UTF-8
assert(!std::uc::is_normalized(std::uc::as_utf32(s)));

char * nfc_s = new char[s.size() * 2];
// Have to use as_utf32(), because normalization operates on code points,
        not UTF-8.
auto    out    =    std::uc::normalize<std::uc::nf::c>(std::uc::as_utf32(s),
        nfc_s);
*out = '\0';
assert(std::uc::is_normalized(nfc_s, out));
```

§ 4.2 Case 2: Normalize a sequence of code points to NFC, where the output is going into a string-like container

This is like the previous case, except that the results must go into a string-like container, not just any old output iterator. The advantage of doing things this way is that the code is a lot faster if you can append the results in chunks.

```
std::string s = /* ... */; // using a std::string to store UTF-8
assert(!std::uc::is_normalized(std::uc::as_utf32(s)));

std::string nfc_s;
nfc_s.reserve(s.size());
// Have to use as_utf32(), because normalization operates on code points,
        not UTF-8.
std::uc::normalize_append<std::uc::nf::c>(std::uc::as_utf32(s), nfc_s);
assert(std::uc::is_normalized(std::uc::as_utf32(nfc_s)));
```

§ 4.3 Case 3: Modify some normalized text without breaking normalization

You cannot modify arbitrary text that is already normalized without risking breaking the normalization. For instance, let’s say I have some NFC-normalized text. That means that

all the combining code points that could combine with one or more preceding code points have already done so. For instance, if I see “ä” in the NFC text, then I know it’s code point U+00E4 “Latin Small Letter A with Diaeresis”, *not* some combination of “a” and a combining two dots.

Now, forget about the “ä” I just gave as an example. Let’s say that I want to insert a single code point, “¨” (U+0308 Combining Diaeresis) into NFC text. Let’s also say that the insertion position is right after a letter “o”. If I do the insertion and then walk away, I would have broken the NFC normalization, because “o” followed by “¨” is supposed to combine to form “ö” (U+00F6 Latin Small Letter O with Diaeresis).

Similar things can happen when deleting text – sometimes the deletion can leave two code points next to each other that should interact in some way that did not apply when they were separate, before the deletion.

```
std::string s = /* ... */;                                    // using a
        std::string to store UTF-8
assert(std::uc::is_normalized(std::uc::as_utf32(s)));      //      already
        normalized

std::string insertion = /* ... */;
normalize_insert<std::uc::nf::c>(s,         s.begin()         +         2,
        std::uc::as_utf32(insertion));
assert(std::uc::is_normalized(std::uc::as_utf32(nfc_s)));
```

§ 5 Proposed design

§ 5.1 Dependencies

This proposal depends on the existence of P2728 “Unicode in the Library, Part 1: UTF Transcoding”.

§ 5.2 Add Unicode version observers

```
namespace std::uc {
  inline constexpr major_version = implementation defined;
  inline constexpr minor_version = implementation defined;
  inline constexpr patch_version = implementation defined;
}
```

Unlike P2728 (Unicode Part 1), the interfaces in this proposal refer to parts of the Unicode standard that are allowed to change over time. The normalization of code points is unlikely to change in Unicode N from what it was for those same code points in Unicode N-1, but since new code points are introduced with each new Unicode release, the normalization algorithms must be updated to keep up.

I’m proposing that implementations provide support for whatever version of Unicode they like, as long as they document which one is supported via `major_`-`/minor_`-`/patch_version`.

§ 5.3 Add stream-safe operations

As mentioned above, I consider most of the Unicode algorithms presented in this proposal and the proposals to come to be low-level tools that most C++ users will not need to touch. I would instead like to see most C++ users use a higher-level, string-like abstraction (provisionally called `std::text`) that will handle all the messy Unicode details, leaving C++ users to think about their program instead of Unicode). As such, most of the interfaces in this proposal assume that their input is in stream-safe format, but they do not enforce that. The exceptions are `normalize_insert`/`-_erase`/`-_replace` algorithms, which are designed to be operations with which something like `std::text` may be built. These interfaces do not assume stream-safe for inserted text, and in fact they put inserted text *into* stream-safe format.

So, if users use something like `std::uc::normalize<std::uc::fc::d>()`, they may know *a priori* that the input is in stream-safe format, or they may not. If they do not, they can use the stream-safe operations to meet the stream-safe precondition of the call to `std::uc::normalize<std::uc::fc::d>()`.

§ 5.3.1 Add stream-safe algorithms

```
namespace std::uc {
  template<utf_iter I, std::sentinel_for<I> S>
    constexpr I stream_safe(I first, S last);

  template<utf_range_like R>
    constexpr range-like-result-iterator<R> stream_safe(R && r);

  template<utf_iter I, sentinel_for<I> S, output_iterator<uint32_t> O>
     constexpr ranges::copy_result<I, O> stream_safe_copy(I first, S last,
         O out);

  template<utf_range_like R, output_iterator<uint32_t> O>
    constexpr ranges::copy_result<range-like-result-iterator<R>, O>
      stream_safe_copy(R && r, O out);

  template<utf_iter I, sentinel_for<I> S>
    constexpr bool is_stream_safe(I first, S last);

  template<utf_range_like R>
    constexpr bool is_stream_safe(R && r);
}
```

`stream_safe()` is like `std::remove_if()` and related algorithms. It writes the stream-safe subset of the given range into the beginning, and leaves junk at the end. It returns the iterator to the first junk element.

Note that *`range-like-result-iterator`*`<R>` comes from P2728. It provides a `ranges::borrowed_iterator_t<R>` or just a pointer, as appropriate, based `R`.

§ 5.3.2 Add `stream_safe_iterator`

```
namespace std::uc {
  constexpr int uc-ccc(uint32_t cp); // exposition only

  template<code_point_iter I, sentinel_for<I> S = I>
  struct stream_safe_iterator
    : iterator_interface<stream_safe_iterator<I, S>, forward_iterator_tag,
        uint32_t, uint32_t> {
    constexpr stream_safe_iterator() = default;
    constexpr stream_safe_iterator(I first, S last)
      : first_(first), it_(first), last_(last),
        nonstarters_(it_ != last_ && uc-ccc(*it_) ? 1 : 0)
        {}

    constexpr uint32_t operator*() const;

    constexpr I base() const { return it_; }

    constexpr stream_safe_iterator& operator++();

          friend constexpr bool operator==(stream_safe_iterator lhs,
        stream_safe_iterator rhs)
      { return lhs.it_ == rhs.it_; }
    template<class I, class S>
       friend constexpr bool operator==(const stream_safe_iterator<I, S>&
        lhs, S rhs)
        { return lhs.base() == rhs; }

    using base_type =  // exposition only
      iterator_interface<stream_safe_iterator<I, S>, forward_iterator_tag,
        uint32_t, uint32_t>;
    using base_type::operator++;

  private:
    I first_;                       // exposition only
    I it_;                          // exposition only
    [[no_unique_address]] S last_; // exposition only
    size_t nonstarters_ = 0;        // exposition only
  };
}
```

uc-ccc() returns the Canonical Combining Class, which indicates how and whether a code point combines with other code points. For some code point `cp`, *uc-ccc*(`cp`) == `0` iff `cp` is a “starter”/“noncombiner”. Any number of “nonstarters”/“combiners” may follow a starter (remember that the purpose of the stream-safe format is to limit the maximum number of combiners to at most 30).

The behavior of this iterator should be left to the implementation, as long as the result meets the stream-safe format, and does not 1) change or remove any starters, or 2) change the first 30 nonstarters after any given starter. The Unicode standard shows a technique for inserting special dummy-starters (that do not interact with most other text) every 30 non-starters, so that the original input is preserved. I think this is silly – the longest possible meaningful sequence of nonstarters is 17 code points, and that is only necessary for backwards comparability. Most meaningful sequences are much shorter. I think a more

reasonable implementation is simply to truncate any sequence of nonstarters to 30 code points.

5.3.3 Add `stream_safe_view` and `as_stream_safe()`

```
namespace std::uc {
  template<class T>
  concept stream-safe-iter = see below;  // exposition only

  template<class I, std::sentinel_for<I> S = I>
    requires stream-safe-iter<I>
  struct stream_safe_view : view_interface<stream_safe_view<I, S>> {
    using iterator = I;
    using sentinel = S;

    constexpr stream_safe_view() {}
    constexpr stream_safe_view(iterator first, sentinel last) :
      first_(first), last_(last)
    {}

    constexpr iterator begin() const { return first_; }
    constexpr sentinel end() const { return last_; }

            friend   constexpr   bool   operator==(stream_safe_view   lhs,
        stream_safe_view rhs)
      { return lhs.first_ == rhs.first_ && lhs.last_ == rhs.last_; }

  private:
    iterator first_;                      // exposition only
    [[no_unique_address]] sentinel last_; // exposition only
  };

   struct as-stream-safe-t : range_adaptor_closure<as-stream-safe-t> { //
        exposition only
    template<utf_iter I, std::sentinel_for<I> S>
      constexpr unspecified operator()(I first, S last) const;
    template<utf_range_like R>
      constexpr unspecified operator()(R && r) const;
  };

  inline constexpr as-stream-safe-t as_stream_safe;
}
```

`stream-safe-iter``<T>` is `true` iff `T` is a specialization of `stream_safe_iterator`.

The `as_stream_safe()` overloads each return a `stream_safe_view` of the appropriate type.

`as_stream_safe(/*...*/)` returns a `stream_safe_view` of the appropriate type, except that the range overload returns `ranges::dangling{}` if `!is_pointer_v<remove_reference_t<R>> && !ranges::borrowed_range<R>` is `true`. If either overload is called with a non-common range `r`, the type of the second template

parameter to `stream_safe_view` will be `decltype(ranges::end(r))`, *not* a specialization of `stream_safe_iterator`.

`as_stream_safe` can also be used as a range adaptor, as in `r | std::uc::as_stream_safe`.

§ 5.4 Add concepts that describe the constraints on parameters to the normalization API

```
namespace std::uc {
  template<class T, class CodeUnit>
  concept eraseable-insertable-sized-bidi-range = // exposition only
    ranges::sized_range<T> &&
    ranges::bidirectional_range<T> &&
    requires(T t, const CodeUnit* it) {
      { t.erase(t.begin(), t.end()) } -> same_as<ranges::iterator_t<T>>;
      { t.insert(t.end(), it, it) } -> same_as<ranges::iterator_t<T>>;
    };

  template<class T>
    concept utf8_string =
      utf8_code_unit<ranges::range_value_t<T>> &&
      eraseable-insertable-sized-bidi-range<T, ranges::range_value_t<T>>;

  template<class T>
    concept utf16_string =
      utf8_code_unit<ranges::range_value_t<T>> &&
      eraseable-insertable-sized-bidi-range<T, ranges::range_value_t<T>>;

  template<class T>
    concept utf_string = utf8_string<T> || utf16_string<T>;
}
```

§ 5.5 Add an enumeration listing the supported normalization forms

`nf` is short for normalization form, and the letter(s) of each enumerator indicate a form. The Unicode normalization forms are NFC, NFD, NFKC, and NFKD. There is also an important semi-official one called FCC (described in Unicode Technical Note #5).

Using this enumeration, a user would spell NFD `std::uc::nf::d`.

```
namespace std::uc {
  enum class nf {
    c,
    d,
    kc,
    kd,
    fcc
  };
```

§ 5.6 Add a generic normalization algorithm

`normalize()` takes some input in code points, and writes the result to the out iterator `out`, also in code points. Since there are fast implementations that normalize UTF-16 and UTF-8 sequences, if the user passes a UTF-8 -> UTF-32 or UTF-16 -> UTF-32 transcoding iterator to `normalize()`, it is allowed to get the underlying iterators out of `[first, last)`, and do all the normalization in UTF-8 or UTF-16.

You may expect `normalize()` to return an alias of `in_out_result`, like `std::ranges::copy()`, or `std::uc::transcode_to_utf8()` from P2728. The reason it does not is that to do so would interfere with using ICU to implement these algorithms. See the section on implementation experience for why that is important.

```
namespace std::uc {
      template<nf   Normalization,   utf_iter   I,   sentinel_for<I>   S,
        output_iterator<uint32_t> O>
    constexpr O normalize(I first, S last, O out);

   template<nf  Normalization,  utf_range_like  R,  output_iterator<uint32_t>
        O>
    constexpr O normalize(R&& r, O out);

  template<nf Normalization, utf_iter I, sentinel_for<I> S>
    constexpr bool is_normalized(I first, S last);

  template<nf Normalization, utf_range_like R>
    constexpr bool is_normalized(R&& r);
}
```

§ 5.7 Add an append version of the normalization algorithm

In performance tests, I found that appending multiple elements to the output in one go was substantially faster than the more generic `normalize()` algorithm, which appends to the output one code point at a time. So, we should provide support for that as well, in the form of `normalize_append()`.

If transcoding is necessary when the result is appended, `normalize_append()` does automatic transcoding to UTF-N, where N is implied by the size of `String::value_type`.

```
namespace std::uc {
   template<nf  Normalization,  utf_iter  I,  sentinel_for<I>  S,  utf_string
        String>
    constexpr void normalize_append(I first, S last, String& s);

  template<nf Normalization, utf_range_like R, utf_string String>
    constexpr void normalize_append(R&& r, String& s);

  template<nf Normalization, utf_string String>
    constexpr void normalize_string(String& s);
}
```

§ 5.8 Add normalization-aware insertion, erasure, and replacement operations on strings

If you need to insert text into a `std::string` or other STL-compatible container, you can use the erase/insert/replace API. There are iterator and range overloads of each. Each one:

— normalizes the inserted text (if text is being inserted);

— places the inserted text in Stream-Safe Format (if text is being inserted);

— performs the erase/insert/replace operation on the string;

— ensures that the result is in Stream-Safe Format (if text is being erased); and

— normalizes the code points on either side of the affected subsequence within the string.

This last step is necessary because insertions and erasures may create situations in which code points which may combine are now next to each other, when they were not before. It's all very complicated, and the user should have a means of doing this generically, and remaining ignorant of the details.

This API is like the `normalize_append()` overloads in that it may operate on UTF-8 or UTF-16 containers, and deduces the output UTF from the size of the mutated container's `value_type`.

About the need for `replace_result`: `replace_result` represents the result of inserting a sequence of code points `I` into an existing sequence of code points `E`, ensuring proper normalization. Since the insertion operation may need to change some code points just before and/or just after the insertion due to normalization, the code points described by `replace_result` may be longer than `I`. `replace_result` values represent the entire sequence of code points in `E` that have changed – some version of which may have already been present in the string before the insertion.

Note that `replace_result::iterator` refers to the underlying sequence, which may not itself be a sequence of code points. For example, the underlying sequence may be a sequence of `char` which is interpreted as UTF-8. We can't return an iterator of the type `I` passed to `normalize_replace()`, for the same reason we don't return an `in_out_result` from `normalization()`.

```
namespace std::uc {
  template<class I>
  struct replace_result : subrange<I> {
    using iterator = I;

    constexpr replace_result() = default;
     constexpr replace_result(iterator first, iterator last) : subrange<I>
         (first, last) {}

    constexpr operator iterator() const { return this->begin(); }
  };
```

```
  enum insertion_normalization {
    insertion_normalized,
    insertion_not_normalized
  };

  template<
    nf Normalization,
    utf_string String,
    code_point_iter I,
    class StringIter = ranges::iterator_t<String>>
  constexpr replace_result<StringIter> normalize_replace(
    String& string,
    StringIter str_first,
    StringIter str_last,
    I first,
    I last,
    insertion_normalization insertion_norm = insertion_not_normalized);

  template<
    nf Normalization,
    utf_string String,
    code_point_iter I,
    class StringIter = ranges::iterator_t<String>>
  constexpr replace_result<StringIter> normalize_insert(
    String& string,
    StringIter at,
    I first,
    I last,
    insertion_normalization insertion_norm = insertion_not_normalized);

  template<
    nf Normalization,
    utf_string String,
    code_point_range R,
    class StringIter = ranges::iterator_t<String>>
  constexpr replace_result<StringIter> normalize_insert(
    String& string,
    StringIter at,
    R&& r,
    insertion_normalization insertion_norm = insertion_not_normalized);

  template<
    nf Normalization,
    utf_string String,
    class StringIter = ranges::iterator_t<String>>
  constexpr replace_result<StringIter> normalize_erase(
    String& string, StringIter str_first, StringIter str_last);
}
```

§ 5.9 Add a feature test macro

Add the feature test macro `__cpp_lib_unicode_normalization`.

§ 6 Implementation experience

All of these interfaces have been implemented in Boost.Text (proposed – not yet a part of Boost). All of the interfaces here have been very well-exercised by full-coverage tests, and by other parts of Boost.Text that use normalization.

The first attempt at implementing the normalization algorithms was fairly straightforward. I wrote code following the algorithms as described in the Unicode standard and its accompanying Annexes, and got all the Unicode-published tests to pass. However, comparing the performance of the naive implementation to the performance of the equivalent ICU normalization API showed that the naive implementation was a *lot* slower – around a factor of 50!

I managed to optimize the initial implementation quite a lot, and got the performance delta down to about a factor of 10. After that, I could shave no more time off of the naive implementation. I looked at how ICU performs normalization, and had a brief discussion about performance with one of the ICU maintainers. It turns out that if you understand the normalization-related Unicode data very deeply, you can take advantage of certain patterns in those data to take shortcuts. In fact, it is only necessary to perform the full algorithm as described in the Unicode standard in a small minority of cases. ICU is maintained in lockstep with the evolution of Unicode, so as new code points are added (often from new languages), the new normalization data associated with those new code points are designed so that they enable the shortcuts mentioned above.

In the end, I looked at the ICU normalization algorithms, and reimplemented them in a generic way, using templates and therefore header-based code. Being generic, this reimplementation works for numerous types of input (iterators, ranges, pointers to null-terminated strings) – not just `icu::UnicodeString` (a `std::string`-like UTF-16 type) and `icu::StringPiece` (a `std::string_view`-like UTF-8 type) that ICU supports. Being inline, header-only code, the reimplementation optimizes better, and I managed about a 20% speedup over the ICU implementation.

However, this reimplementation of ICU was a lot of work, and there's no guarantee that it will work for more than just the current version of Unicode supported by Boost.Text. Since ICU and Unicode evolve in lockstep, any reimplementation needs to track changes to the ICU implementation when the Unicode version is updated, and the equivalent change needs to be applied to the reimplementation.

§ 6.1 tl;dr

Standard library implementers will probably want to just use ICU to implement the normalization algorithms. Since ICU only implements the normalization algorithms for UTF-16 and UTF-8, and since it only implements the algorithms for the exact types `icu::UnicodeString` (for UTF-16) and `icu::StringPiece` (for UTF-8), copying may need to occur. There are implementation-detail interfaces within ICU that more intrepid

implementers may wish to use; these interfaces can be made to work with iterators and pointers more directly.